

Bright Future Technologies Application

Project Overview

A Java console application for Extreme IT Products to manage a product catalogue. The application supports product capture, search, update, deletion, and reporting.

Project Structure

File	Purpose
Main.java	Application entry point and welcome screen.
Products.java	Menu, product capture, search, update, delete, report and exit functions.
Product.java	Represents an individual product and its properties.
ReportData.java	Stores report summary values and provides get/set methods.

Assignment Requirements

Requirement	Implementation
Q.1.1 Main menu	Main.java and Products.DisplayMenu()
Q.1.2 Capture product	Products.CaptureProduct() and SaveProduct()
Q.1.4 / Q.1.5 Category validation	getCategoryName() and validation loop
Q.1.6 Warranty options	CaptureProduct() and UpdateProduct()
Q.1.7 Success confirmation	CaptureProduct() confirmation message
Q.1.8 Search product	SearchProduct()
Q.1.9 Delete product	DeleteProduct() with confirmation
Q.1.10 Update product	UpdateProduct()
Q.1.11 Product report	PrintReport() and ReportData.java
Q.1.12 Exit application	ExitApplication()

Main.java

```
import java.util.Scanner;

/**
 * Main class used to run the Bright Future Technologies / Extreme IT
 * Products application (Q.1.1).
 */
public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("BRIGHT FUTURE TECHNOLOGIES APPLICATION");
        System.out.println("*****");
        System.out.print("Enter (1) to launch menu or any other key to exit: ");
        String choice = scanner.nextLine().trim();

        if (choice.equals("1")) {
            Products products = new Products(scanner);
            products.DisplayMenu();
        } else {
            System.out.println("Goodbye!");
        }

        scanner.close();
    }
}
```

Products.java

```
import java.util.ArrayList;
import java.util.Scanner;

/**
 * Contains all the working methods for the Bright Future Technologies /
 * Extreme IT Products application, as required by the assignment brief.
 */
public class Products {

    private ArrayList<Product> productList;
    private Scanner scanner;

    public Products(Scanner scanner) {
        this.scanner = scanner;
        this.productList = new ArrayList<>();
    }

    /**
     * Q.1.1 - Displays the main menu and routes the user's selection to the
     * correct method. Loops until the user chooses to exit.
     */
    public void DisplayMenu() {
        boolean keepRunning = true;

        while (keepRunning) {
            System.out.println();
            System.out.println("Please select one of the following menu items:");
            System.out.println("(1) Capture a new product.");
            System.out.println("(2) Search for a product.");
            System.out.println("(3) Update a product.");
            System.out.println("(4) Delete a product.");
            System.out.println("(5) Print report.");
            System.out.println("(6) Exit Application.");
            System.out.print("Selection >> ");
            String choice = scanner.nextLine().trim();

            switch (choice) {
                case "1":
                    CaptureProduct();
                    break;
                case "2":
                    SearchProduct();
                    break;
                case "3":
                    UpdateProduct();
                    break;
                case "4":
                    DeleteProduct();
                    break;
                case "5":
                    PrintReport();
                    break;
                case "6":
                    ExitApplication();
                    keepRunning = false;
                    break;
                default:
                    System.out.println("Invalid selection. Please try again.");
                    continue;
            }

            if (keepRunning) {
                System.out.print("Enter (1) to launch menu or any other key to exit: ");
                String cont = scanner.nextLine().trim();
                if (!cont.equals("1")) {
                    keepRunning = false;
                    System.out.println("Goodbye!");
                }
            }
        }
    }

    /**
     * Q.1.2, Q.1.4, Q.1.5, Q.1.6, Q.1.7 - Captures a new product from the
     * user, validating the product category, and saves it into memory.
     */
    public void CaptureProduct() {
        System.out.println();
    }
}
```

```

System.out.println("CAPTURE A NEW PRODUCT");
System.out.println("*****");

System.out.print("Enter the product code: ");
String code = scanner.nextLine().trim();

System.out.print("Enter the product name: ");
String name = scanner.nextLine().trim();

String category = null;
while (category == null) {
    System.out.println("Select the product category:");
    System.out.println("Desktop Computer - 1");
    System.out.println("Laptop - 2");
    System.out.println("Tablet - 3");
    System.out.println("Printer - 4");
    System.out.println("Gaming Console - 5");
    System.out.print("Product Category >> ");
    String categoryChoice = scanner.nextLine().trim();
    category = getCategoryName(categoryChoice);
    if (category == null) {
        System.out.println("Invalid product category. Please re-enter a valid product category.");
    }
}

System.out.print("Indicate the product warranty. Enter (1) for 6 months or any other key for 2 years. ");
String warrantyChoice = scanner.nextLine().trim();
String warranty = warrantyChoice.equals("1") ? "6 months" : "2 years";

System.out.print("Enter the price for " + name + " >> ");
double price = readDouble();

System.out.print("Enter the stock level for " + name + " >> ");
int stockLevel = readInt();

System.out.print("Enter the supplier for " + name + " >> ");
String supplier = scanner.nextLine().trim();

Product product = new Product(code, name, category, warranty, price, stockLevel, supplier);
SaveProduct(product);

System.out.println("Product details has been saved successfully!!!");
}

/**
 * Saves a captured product into memory (the productList ArrayList).
 */
public void SaveProduct(Product product) {
    productList.add(product);
}

/**
 * Q.1.8 - Searches for a product by product code and displays its
 * details, or an error message if it cannot be found.
 */
public void SearchProduct() {
    System.out.println();
    System.out.print("Please enter the product code to search: ");
    String code = scanner.nextLine().trim();

    Product found = findProduct(code);

    if (found != null) {
        System.out.println("*****");
        System.out.println("PRODUCT SEARCH RESULTS");
        System.out.println("*****");
        printProductDetails(found);
        System.out.println("*****");
    } else {
        System.out.println("The product cannot be located. Invalid Product");
    }
}

/**
 * Q.1.9 - Deletes a product, after the user confirms the deletion.
 */
public void DeleteProduct() {
    System.out.println();
    System.out.print("Please enter the product code to delete: ");
    String code = scanner.nextLine().trim();

```

```

        Product found = findProduct(code);

        if (found == null) {
            System.out.println("The product cannot be located. Invalid Product");
            return;
        }

        System.out.print("Are you sure you want to delete this product? (y) Yes, (n) No: ");
        String confirm = scanner.nextLine().trim();

        if (confirm.equalsIgnoreCase("y")) {
            productList.remove(found);
            System.out.println("Product details has been deleted successfully!!!");
        } else {
            System.out.println("Delete cancelled.");
        }
    }

    /**
     * Q.1.10 - Updates the warranty, price, and/or stock level of an
     * existing product, based on the user's choices.
     */
    public void UpdateProduct() {
        System.out.println();
        System.out.print("Please enter the product code to update: ");
        String code = scanner.nextLine().trim();

        Product found = findProduct(code);

        if (found == null) {
            System.out.println("The product cannot be located. Invalid Product");
            return;
        }

        System.out.print("Update the warranty? (y) Yes, (n) No: ");
        String updateWarranty = scanner.nextLine().trim();
        if (updateWarranty.equalsIgnoreCase("y")) {
            System.out.print("Indicate the product warranty. Enter (1) for 6 months or any other key for 2 years. ");
            String warrantyChoice = scanner.nextLine().trim();
            found.setWarranty(warrantyChoice.equals("1") ? "6 months" : "2 years");
        }

        System.out.print("Update the product price? (y) Yes, (n) No: ");
        String updatePrice = scanner.nextLine().trim();
        if (updatePrice.equalsIgnoreCase("y")) {
            System.out.print("Enter the new price for " + found.getProductName() + " >> ");
            found.setPrice(readDouble());
        }

        System.out.print("Update the stock level? (y) Yes, (n) No: ");
        String updateStock = scanner.nextLine().trim();
        if (updateStock.equalsIgnoreCase("y")) {
            System.out.print("Enter the new stock level for " + found.getProductName() + " >> ");
            found.setStockLevel(readInt());
        }

        System.out.println("Product details has been updated successfully!!!");
    }

    /**
     * Q.1.11 - Prints a full product report, using a ReportData object to
     * hold the summary totals (count, total value, average value).
     */
    public void PrintReport() {
        System.out.println();
        System.out.println("PRODUCT REPORT");
        System.out.println("=====");

        ReportData reportData = new ReportData();
        double totalValue = 0.0;

        for (int i = 0; i < productList.size(); i++) {
            Product p = productList.get(i);
            System.out.println("PRODUCT " + (i + 1));
            System.out.println("-----");
            printProductDetails(p);
            System.out.println("-----");
            totalValue += p.getPrice();
        }

        reportData.setTotalProductCount(productList.size());
    }

```

```

        reportData.setTotalProductValue(totalValue);
        reportData.setAverageProductValue(
            productList.isEmpty() ? 0.0 : totalValue / productList.size());

        System.out.println("=====");
        System.out.println("TOTAL PRODUCT COUNT: " + reportData.getTotalProductCount());
        System.out.printf("TOTAL PRODUCT VALUE: R %.1f%n", reportData.getTotalProductValue());
        System.out.printf("AVERAGE PRODUCT VALUE: R %.0f%n", reportData.getAverageProductValue());
        System.out.println("=====");
    }

    /**
     * Q.1.12 - Exits the application.
     */
    public void ExitApplication() {
        System.out.println("Thank you for using the Bright Future Technologies Application. Goodbye!");
    }

    // ----- Helper methods -----

    private Product findProduct(String code) {
        for (Product p : productList) {
            if (p.getProductCode().equalsIgnoreCase(code)) {
                return p;
            }
        }
        return null;
    }

    private void printProductDetails(Product p) {
        System.out.println("PRODUCT CODE >> " + p.getProductCode());
        System.out.println("PRODUCT NAME >> " + p.getProductName());
        System.out.println("PRODUCT CATEGORY >> " + p.getCategory());
        System.out.println("PRODUCT WARRANTY >> " + p.getWarranty());
        System.out.println("PRODUCT PRICE >> R " + p.getPrice());
        System.out.println("PRODUCT LEVEL >> " + p.getStockLevel());
        System.out.println("PRODUCT SUPPLIER >> " + p.getSupplier());
    }

    private String getCategoryName(String choice) {
        switch (choice) {
            case "1":
                return "Desktop Computer";
            case "2":
                return "Laptop";
            case "3":
                return "Tablet";
            case "4":
                return "Printer";
            case "5":
                return "Gaming Console";
            default:
                return null;
        }
    }

    private double readDouble() {
        while (true) {
            String input = scanner.nextLine().trim();
            try {
                return Double.parseDouble(input);
            } catch (NumberFormatException e) {
                System.out.print("Invalid number, please try again >> ");
            }
        }
    }

    private int readInt() {
        while (true) {
            String input = scanner.nextLine().trim();
            try {
                return Integer.parseInt(input);
            } catch (NumberFormatException e) {
                System.out.print("Invalid number, please try again >> ");
            }
        }
    }
}

```

Product.java

```
/**
 * Represents a single product sold by Extreme IT Products.
 */
public class Product {

    private String productCode;
    private String productName;
    private String category;    // e.g. "Laptop", "Desktop Computer" ...
    private String warranty;    // "6 months" or "2 years"
    private double price;
    private int stockLevel;
    private String supplier;

    public Product(String productCode, String productName, String category,
                   String warranty, double price, int stockLevel, String supplier) {
        this.productCode = productCode;
        this.productName = productName;
        this.category = category;
        this.warranty = warranty;
        this.price = price;
        this.stockLevel = stockLevel;
        this.supplier = supplier;
    }

    public String getProductCode() {
        return productCode;
    }

    public void setProductCode(String productCode) {
        this.productCode = productCode;
    }

    public String getProductName() {
        return productName;
    }

    public void setProductName(String productName) {
        this.productName = productName;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public String getWarranty() {
        return warranty;
    }

    public void setWarranty(String warranty) {
        this.warranty = warranty;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public int getStockLevel() {
        return stockLevel;
    }

    public void setStockLevel(int stockLevel) {
        this.stockLevel = stockLevel;
    }

    public String getSupplier() {
        return supplier;
    }

    public void setSupplier(String supplier) {
        this.supplier = supplier;
    }
}
```

} }

ReportData.java

```
/**
 * Holds the summary values displayed at the bottom of the product report
 * (Q.1.11). Provides a get and set method for each item required in the
 * report, as specified in the assignment.
 */
public class ReportData {

    private int totalProductCount;
    private double totalProductValue;
    private double averageProductValue;

    public ReportData() {
        this.totalProductCount = 0;
        this.totalProductValue = 0.0;
        this.averageProductValue = 0.0;
    }

    public int getTotalProductCount() {
        return totalProductCount;
    }

    public void setTotalProductCount(int totalProductCount) {
        this.totalProductCount = totalProductCount;
    }

    public double getTotalProductValue() {
        return totalProductValue;
    }

    public void setTotalProductValue(double totalProductValue) {
        this.totalProductValue = totalProductValue;
    }

    public double getAverageProductValue() {
        return averageProductValue;
    }

    public void setAverageProductValue(double averageProductValue) {
        this.averageProductValue = averageProductValue;
    }
}
```